

LogicConnect: Structured Connectivity Design for Differentiable Logic Gate Networks

Abstract—Differentiable logic gate networks learn Boolean circuits whose connectivity determines which signals can be combined during gate-function optimization. Learning these connections can improve accuracy, but increases the memory required to process candidate inputs during training. In this paper, we present LogicConnect, a structured connectivity method that constructs the graph before training while preserving the network’s gate budget. LogicConnect combines semantic input pairing with regular multiscale matchings in deeper layers and supports dense and convolutional architectures without trainable routing parameters. We evaluate the method on MNIST, Fashion-MNIST, and CIFAR-10 through accuracy comparisons, training-resource measurements, and structural ablations. LogicConnect improves dense CIFAR-10 test accuracy by 3.18–4.59 percentage points across three model sizes and three paired seeds. Convolutional S/M models improve by 3.26/2.08 points in single-seed tests, although the three-seed S validation comparison remains inconclusive. Compared with learned Top-32 routing, LogicConnect uses 93.7–94.2% less peak GPU memory and 80% fewer trainable parameters, with an accuracy reduction of 1.84–5.30 points. Degree-preserving ablations identify first-layer pairing as the strongest measured dense contribution, supporting structured connectivity as a design choice for improving accuracy under constrained training memory.

I. INTRODUCTION

Differentiable Logic Gate Networks (DLGNs) provide an alternative path toward efficient machine learning at the edge by learning directly with the primitive operations used in digital hardware. Rather than relying on multiply-accumulate operations, a DLGN is composed of Boolean gates. During training, these discrete operations are represented through differentiable relaxations, allowing the network to be optimized with gradient descent. After training, each relaxed gate is hardened to a Boolean function for efficient inference [1]. This direct correspondence between the learned model and Boolean computation makes DLGNs particularly attractive for resource-constrained embedded and edge and systems.

However, a fundamental characteristic of DLGNs is their sparse connectivity. Each conventional DLGN gate has only two inputs. These two predecessors are selected before training and remain fixed, while optimization determines which Boolean function the gate should implement [1]. Therefore, training can change *how* two signals are combined, but not *which* signals arrive at a gate. To that end, if two informative signals are never connected through the available graph, learning the gate

functions alone cannot create that missing edge. At a fixed depth and width, connectivity consequently determines the combinations of information that can be formed throughout the network and becomes part of the effective architectural capacity of the DLGN.

A way to address this issue is to consider connectivity as an independent design dimension [2] and optimize the predecessors themselves during training [3], [4]. This type of learned connectivity can recover connections that were absent from the original graph and can therefore improve accuracy. However, such methods require significantly increased training time as each gate must consider multiple candidate signals, requiring additional routing state and candidate activations even though the final hardened network may retain only a small number of predecessors. This creates a practical design choice between inexpensive but arbitrary *fixed random connectivity* and more flexible but substantially heavier *learned connectivity*.

To demonstrate the need for connectivity design, we consider a motivational example on a dense CIFAR-10 DLGN. We keep the deeper connectivity unchanged and randomize only the first-layer connections under a degree-preserving constraint. Hence, every predecessor is used exactly the same number of times before and after randomization; only the identities of the signals paired at each gate are changed. Despite preserving the same fan-out, hardened validation accuracy decreases by 4.49 percentage points. This result reveals an important limitation of degree-balanced connectivity: *how often a signal is used does not determine which information is brought together*. However, explicitly selecting favorable pairs is itself challenging to scale. Given N candidate predecessors, a two-input gate can already choose from $O(N^2)$ possible pairs, and this choice is repeated across thousands or millions of gates and multiple network layers. Moreover, while input signals have explicit image semantics, intermediate signals progressively lose this direct interpretation, making a manually defined pairing rule increasingly difficult to justify at deeper layers. Fixed random connectivity avoids this search but ignores signal relationships, whereas learned connectivity explores the enlarged space by maintaining additional routing state and candidate activations during training. Therefore, a key challenge is to construct informative connections *before training* using principles that remain scalable as the network grows and do not introduce the training overhead of a separate routing problem.

In this paper, we propose **LogicConnect**, an offline connectivity construction that introduces structure into the DLGN graph before training. LogicConnect first exploits signal identity where it is most explicit by forming semantic first-layer pairs

according to relationships among the encoded inputs rather than arbitrary indices. In subsequent layers, it organizes predecessor interactions through regular multiscale matchings, selecting complete matching stages primarily to balance predecessor usage and using ancestry overlap to distinguish alternatives with equivalent degree balance. Our main contributions are:

II. RELATED WORK

Petersen et al. introduced Differentiable Logic Gate Networks (DLGNs), enabling gradient-based optimization of Boolean gate functions while keeping randomly initialized connections fixed during training [1]. In [5], the authors extended DLGNs to convolutional architectures, exploiting image structure through shared logic trees, spatial sampling, pooling, and residual initialization. Other works extend DLGNs to recurrent models [6], improve output aggregation for larger classification problems [7], or develop interpretable and truth-table-based logic architectures [8]–[10]. Related differentiable logic models have also been explored for cellular automata [?], [11] and circuit representation learning [12]. However, these works primarily focus on extending the architecture and applicability of logic-based networks, without exploring the connectivity problem.

Regarding DLGN connectivity, Mommen et al. optimize distributions over candidate predecessors, allowing gate inputs to change during training [3]. Similarly, LILogic Net employs sparse Top-K routing to learn compact network connectivity [4], while BitLogic studies connectivity together with encoding, fan-in, node parameterization, and output-head design [2]. These works demonstrate that selecting better predecessors can significantly improve logic networks. However, learned routing requires processing candidate connections and maintaining additional routing parameters and intermediate activations during training, increasing both execution time and memory requirements, even though the hardened network ultimately retains only a small number of inputs per gate. In contrast, our approach constructs the connectivity *offline* and keeps it fixed during gate training, avoiding an additional routing optimization problem while preserving the declared depth, width, and gate budget.

Additionally, several works focus on improving the optimization of the gate functions themselves. Mind the Gap reduces the discrepancy between continuous training and discrete inference [?]. Light DLGNs redesign the gate parameterization to improve training efficiency [13], while WARP introduces a Walsh-based Boolean representation [14]. The works in [15], [16] further explore polynomial formulations to analyze or replace conventional soft gate mixtures. These methods optimize *what function* a gate computes from its assigned inputs, whereas our work focuses on *which signals* are made available to that gate.

III. LOGICCONNECT METHODOLOGY

LogicConnect uses the input representation and layer dimensions to construct connections before training, as shown in Figure 1. Semantic pairing exploits known relationships between encoded image values, while multiscale matching extends the construction to hidden signals whose spatial meaning is no longer explicit. Both components produce fixed predecessor indices, allowing training with the original gate parameterization.

A. Network representation

For a layer l containing m_l two-input gates and n_l available predecessors, we represent connectivity by $I_l \in \{0, \dots, n_l - 1\}^{2 \times m_l}$. Gate j receives distinct predecessors $a = I_l[0, j]$ and $b = I_l[1, j]$. Following the baseline DLGN formulation [1], its relaxed output is

$$h_{l,j} = \sum_{q=0}^{15} \text{softmax}(\theta_{l,j})_q \tilde{f}_q(h_{l-1,a}, h_{l-1,b}). \quad (1)$$

Here, $\theta_{l,j}$ contains the trainable gate logits and $\tilde{f}_q$ denotes the continuous relaxation of Boolean function q . Training optimizes θ on the fixed graph, and discretization selects the highest-weight function for each gate. LogicConnect determines I_l while preserving this optimization, the loss function, and output aggregation.

We associate each predecessor i with a structural ancestry set A_i describing its accessible sources. At a dense image input, labels identify raw channel-pixel values, with thermometer thresholds of the same value sharing a label. Propagating $A_j = A_a \cup A_b$ records sources that can reach each gate through its connections. The learned gate function determines which accessible inputs actually affect the output.

B. Semantic input pairing

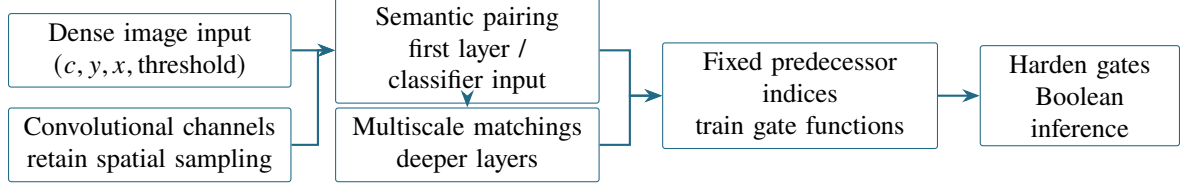
The image encoding provides useful information that is lost when inputs are treated only as numbered bits. In particular, thresholds of the same pixel-channel value share a source, whereas bits from different positions or channels can convey distinct image information. We use this distinction to construct candidate pairs from channel, spatial, and threshold metadata, excluding pairs whose two bits come from the same raw source.

Spatial candidates connect horizontal and vertical positions at power-of-two strides up to half the corresponding axis length, using cyclic boundaries. Additional candidates at the finest scale connect aligned channels and combine different thresholds with a spatial displacement. The construction cycles through these candidate stages, distributing connections across the available relationships while controlling how often each predecessor is selected.

Within each stage, candidates are ordered by the sum of endpoint fan-outs, then their maximum fan-out, and finally their indices. The initial selection examines up to 64 unused candidates in a seeded affine order; once distinct pairs are exhausted, repeated pairs follow the same degree priority. This bounded search preserves a deterministic construction while limiting the cost of selecting semantically defined pairs.

C. Multiscale matching in hidden layers

Deeper signals lack a direct image-coordinate interpretation, so we construct their connections through regular matchings over predecessor indices and use ancestry to distinguish their accessible sources. Each matching contains disjoint pairs, allowing a predecessor to participate at most once per stage. Varying the matching changes which signals are combined while controlling predecessor reuse.



Stage priority: degree spread $\rightarrow$ ancestry novelty $\rightarrow$ nominal scale

Fig. 1: Overview of LogicConnect. Semantic pairing constructs input connections, and multiscale matching organizes deeper connections. The convolutional construction preserves spatial sampling and shared logic trees. Training optimizes gate functions on the resulting fixed graph.

For n predecessors, the construction considers $K = \lceil \log_2 n \rceil$ scales. When n is a power of two, scale s pairs index i with $i \oplus 2^s$, providing connections at different index separations. For other widths, an affine traversal with a coprime step pairs adjacent elements; at odd widths, the unmatched predecessor rotates. Each complete stage contains $\lfloor n/2 \rfloor$ pairs. If fewer gates remain, a partial stage selects disjoint pairs by endpoint degree sum, maximum degree, and index.

We select stages by prioritizing balanced reuse and then considering the overlap between their ancestry sets. Let d_i be the current fan-out of predecessor i , and let $\delta_i(P_s)$ count its occurrences in the pairs retained from candidate stage s . The resulting degree spread is

$$D_s = \max_i (d_i + \delta_i(P_s)) - \min_i (d_i + \delta_i(P_s)). \quad (2)$$

Minimizing D_s gives preference to stages that maintain uniform access to preceding signals. Among candidates with the same spread, we favor pairs with less relative ancestry overlap, measured by

$$N_s = \frac{1}{|Q_s|} \sum_{(a,b) \in Q_s} \left(1 - \frac{|A_a \cap A_b|}{\max(1, \min(|A_a|, |A_b|))} \right). \quad (3)$$

The set Q_s contains the complete candidate matching, or 2,048 evenly indexed pairs when the matching is larger. We use the same set when selecting a partial stage. Identical nonempty ancestry sets contribute zero to this score, while disjoint sets contribute one. LogicConnect minimizes D_s and then maximizes N_s , resolving remaining ties by cyclic nominal scale order. Every scale is used once per cycle, as summarized in Algorithm 1. Ancestry overlap is a structural heuristic whose contribution to accuracy is evaluated separately.

For even widths, complete matchings increment every degree equally, and a final partial matching increments each degree at most once. Starting from uniform degrees therefore gives a final spread of at most one. For example, eight predecessors feeding ten gates use two complete matchings and two additional pairs, producing degrees of two or three. Degree-preserving ablations test the accuracy contribution of pair identities beyond this balance property.

D. Extension to convolutional networks

Convolutional DLGNs share logic trees across spatial positions, requiring the construction to preserve both channel

Algorithm 1 Deeper-layer stage selection

Require: Ancestry A , width m , layer l , seed

Initialize degrees $d = 0$, output pairs $I = \emptyset$.

while $|I| < m$ **do**

 Generate the K seeded scale matchings M_s .

 Form degree-ordered partial stages P_s if needed.

 Compute D_s and N_s using (2)–(3).

 Among unused scales, minimize D_s , then maximize N_s ; break ties by nominal scale order.

 Append P_s ; update d and mark scale s used.

 Reset used scales after a complete K -stage cycle.

end while

Propagate ancestry by union; return fixed indices I .

organization and spatial sampling [5]. We apply LogicConnect to channel pairing while retaining the original receptive-field sampler, depth-three logic trees, weight sharing, and OR pooling. The first channel assignment uses semantic metadata, and subsequent channel groups use multiscale matching. This formulation changes connectivity at the channel level without replacing the convolutional computation with independent dense gates.

Channel ancestry initially uses encoded-channel identities. We add an identity label to each output channel so that channels remain distinguishable when their upstream support sets become identical. After flattening, classifier inputs retain this ancestry and receive spatial-feature identities; semantic pairing constructs the classifier input connections, followed by deeper matching stages. These labels support a common construction at different signal granularities, rather than representing exact pixel-dependence sets.

E. Construction overhead

Construction uses packed ancestry bitsets; CPU storage grows with layer width and label universe. Each stage decision considers K matchings and evaluates at most 2,048 ancestry pairs per candidate. We measure this one-time cost separately from training. A dense layer retains $16m_l$ trainable gate logits and $2m_l$ fixed indices, discarding ancestry before deployment. The circuit retains its logical budget; physical area and interconnect cost depend on synthesis and placement.

TABLE I: Main evaluation configurations. b : thresholds per raw value; K denotes 1,000. Dense widths enumerate gates per layer; convolutional widths enumerate output channels with $k = 32/256$ for S/M.

Dataset/model	Layer widths	Gate functions	b	Updates	Batch
MNIST	$5 \times 1,334 + 1,330$	8K	1	108K	100
Fashion-MNIST	$5 \times 2,666 + 2,670$	16K	3	108K	100
CIFAR-10 S	$4 \times 12K$	48K	3	108K	100
CIFAR-10 M	$4 \times 128K$	512K	3	108K	100
CIFAR-10 L	$5 \times 256K$	1,280K	5	108K	100
Conv. S/M	$(k, 4k, 16k, 32k)$	83,552 / 668,416	3	350K / 200K	128

Dense: Adam, learning rate 0.01, no augmentation. Convolutional: AdamW, learning rate 0.02, decay 0.002, random crops with four-pixel padding and horizontal flips. All use raw gate logits and validation every 2K updates. Convolutional S/M share gate functions across 874,496/6,995,968 spatial gate applications; each classifier uses three dense layers of widths $(1280k, 640k, 320k)$.

TABLE II: Dense test accuracy (T, %): mean $\pm$ sample SD over three paired seeds. Δ LC denotes the percentage-point gain over random connectivity.

Model	Gates, K	Random (REP)	LC (OUR)	LC-D (OUR)	Δ LC	Paired 95% CI	Random (R)
MNIST	8	91.27 $\pm$ 0.22	91.94 $\pm$ 0.14	91.91 $\pm$ 0.31	+0.66	[0.02, 1.31]	—
Fashion-MNIST	16	85.20 $\pm$ 0.26	85.72 $\pm$ 0.45	85.91 $\pm$ 0.36	+0.52	[−1.02, 2.06]	—
CIFAR-10 S	48	48.91 $\pm$ 0.35	52.10 $\pm$ 0.63	52.39 $\pm$ 0.30	+3.18	[0.77, 5.60]	51.27 $\pm$ 0.26
CIFAR-10 M	512	54.10 $\pm$ 0.17	58.65 $\pm$ 0.17	58.36 $\pm$ 0.25	+4.56	[3.78, 5.33]	57.39 $\pm$ 0.13
CIFAR-10 L	1,280	55.87 $\pm$ 0.31	60.46 $\pm$ 0.35	61.07 $\pm$ 0.46	+4.59	[3.72, 5.47]	60.78 $\pm$ 0.12

LC improves in all three seed pairs for every model. LC-D: dense specialization. R: published baseline [1]; exact-budget MNIST/Fashion references are unavailable.

REFERENCES

- [1] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, “Deep differentiable logic gate networks,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 2006–2018, 2022.
- [2] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, “BitLogic: A framework for gradient-based LUT-native neural networks,” *Transactions on Machine Learning Research*, Jun. 2026.
- [3] W. Mommen, L. Keuninckx, M. Hartmann, and P. Wambacq, “A method for optimizing connections in differentiable logic gate networks,” *arXiv preprint arXiv:2507.06173*, 2025.
- [4] K. Fojcik, R. Zioma, and J. Armitis, “Lillogic net: Compact logic gate networks with learnable connectivity for efficient hardware deployment,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 3498–3507.
- [5] F. Petersen, H. Kuehne, C. Borgelt, J. Welzel, and S. Ermon, “Convolutional differentiable logic gate networks,” *Advances in neural information processing systems*, vol. 37, pp. 121 185–121 203, 2024.
- [6] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, “Recurrent deep differentiable logic gate networks,” in *Proceedings of the 2nd International Workshop on Edge and Mobile Foundation Models*, ser. EdgeFM ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 31–36.
- [7] S. Brändle, T. Aczel, A. Plesner, and R. Wattenhofer, “From mnist to imagenet: Understanding the scalability boundaries of differentiable logic gate networks,” 2025.
- [8] C. Yue and N. K. Jha, “Learning interpretable differentiable logic networks,” *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, vol. 1, no. 1, pp. 69–82, 2024.
- [9] A. Benamira, T. Guérard, T. Peyrin, T. Yap, and B. Hooi, “A scalable, interpretable, verifiable & differentiable logic gate convolutional neural network architecture from truth tables,” 2023.
- [10] A. Benamira, T. Peyrin, T. Yap, T. Guérard, and B. Hooi, “Truth table net: Scalable, compact & verifiable neural networks with a dual convolutional small boolean circuit networks form.” in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 13–21.
- [11] P. Miotti, E. Niklasson, E. Randazzo, and A. Mordvintsev, “Differentiable logic cellular automata: From game of life to pattern generation,” in *Artificial Life Conference Proceedings 37*, vol. 2025, no. 1. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info . . . , 2025, p. 54.
- [12] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu, “Deepgate: learning neural representations of logic gates,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC ’22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 667–672.
- [13] L. Rüttgers, T. Aczel, A. Plesner, and R. Wattenhofer, “Light differentiable logic gate networks,” in *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, Eds., vol. 2026, 2026, pp. 66 744–66 776.
- [14] L. Gerlach, T. Gerlach, L. Våge, E. Kauffman, and I. Ojalvo, “Warp logic neural networks,” 2026.
- [15] Y. Kim, “Fitting multilinear polynomials for logic gate networks,” 2026.
- [16] S. S. Damera, R. Matheu, A. Puranic, and J. S. Baras, “Polynomial surrogate training for differentiable ternary logic gate networks,” *ArXiv*, vol. abs/2603.00302, 2026.